

TypeScript TCP Chat

Study Notes, Architecture and Debugging Review

A condensed reference with deeper explanations for the hard parts

Project outcome: A working multi-client TCP chat in TypeScript/Node.js with Login/Register, PBKDF2 password hashing, persistent users.json storage, newline-delimited JSON, Join/Leave/Chat broadcasts, async stream reading and clean separation between client, server and shared message contracts.

Focus of this guide: what you built, what each layer does, where you struggled, why those errors happened, and the debugging rules worth carrying into REST APIs and React.

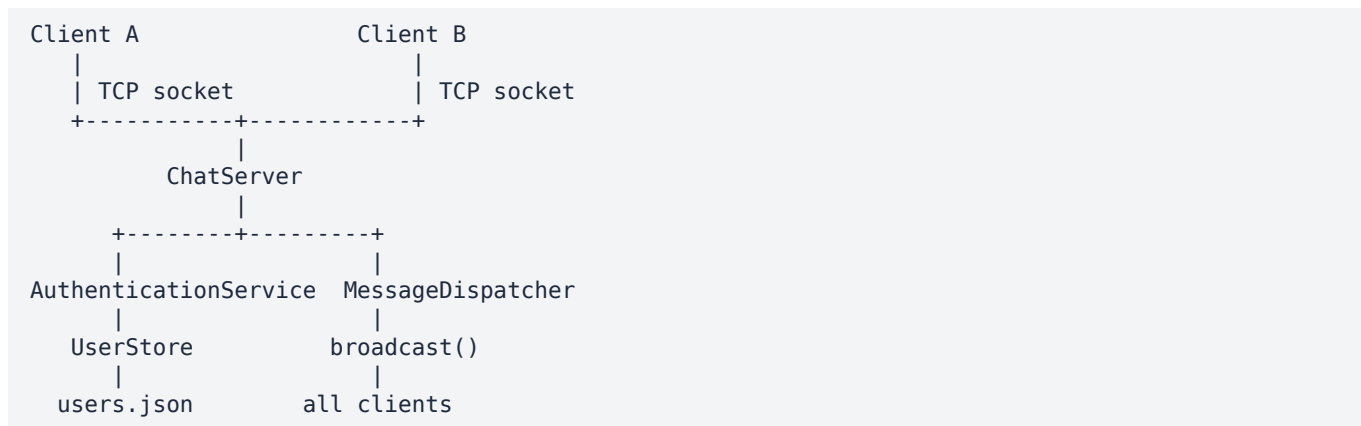
Contents

1. Project overview and architecture
2. File structure and responsibility map
3. Shared message types and discriminated unions
4. TCP byte streams, framing and LineReader
5. ClientConnection and socket lifecycle
6. ChatClient lifecycle: connect, authenticate, send, receive
7. AuthenticationService, PBKDF2 and UserStore
8. ChatServer orchestration and MessageDispatcher
9. TypeScript syntax that mattered most
10. Your main struggles and what each one taught you
11. Debugging stack traces and compiler cascades
12. Final end-to-end data flow
13. Memory rules and next-step practice

How to use this: Read sections 1-8 once for architecture. Revisit sections 9-11 whenever TypeScript syntax or compiler errors feel unclear. The final pages are designed as a quick reference before your next Node REST API.

1. Project overview and architecture

This project recreated the core behavior of your earlier C# TCP chat in TypeScript and Node.js. The important learning was not the terminal UI; it was the transport, message framing, authentication, persistence and async control flow underneath it.



Layer	Responsibility	C# mental mapping
ChatClient	Connects, authenticates, reads console input, receives server messages	TcpClient + client loop
ChatServer	Accepts sockets and coordinates the session lifecycle	TcpListener + server orchestration
ClientConnection	Wraps one socket and line-based send/receive behavior	TcpClient/NetworkStream wrapper
LineReader	Converts arbitrary TCP chunks into complete newline-delimited messages	StreamReader.ReadLineAsync style behavior
AuthenticationService	Register/login decision and credential verification	Application/service layer
UserStore	Reads/writes persisted user accounts	Repository/file storage layer
MessageDispatcher	Broadcasts Join/Leave/Chat messages	Broadcast/message distribution service
messages.ts	Shared contracts between client and server	DTOs/contracts

Core architectural lesson: The server decides what happened; the dispatcher distributes it; the connection writes it; the client displays it. Keeping these responsibilities separate made the system easier to reason about and debug.

2. File structure and responsibility map

```

src/
  client/

```

```
chatClient.ts
consoleInput.ts
main.ts

server/
  authenticationService.ts
  chatServer.ts
  clientConnection.ts
  main.ts
  messageDispatcher.ts
  passwordHasher.ts
  userStore.ts

shared/
  jsonLines.ts
  messages.ts

data/
  users.json
```

The separation is intentional. Files in `shared/` describe the wire protocol and framing utilities used by both sides. The client should not import server-only classes such as `ChatServer` or `UserStore`. The server should own authentication and persistence.

Why this matters for REST APIs: The same separation appears later as routes/controllers → services → repositories/database, with shared DTO/schema types at the boundary.

3. Shared message types and discriminated unions

The chat protocol uses TypeScript unions so the compiler knows the valid message shapes. The type field is the discriminator: it tells your code which member of the union it is dealing with.

```
export type AuthType = "Login" | "Register";
export type ChatMessageType = "Chat" | "Join" | "Leave";

export interface AuthRequest {
  type: AuthType;
  userName: string;
  password: string;
}

export interface ChatSendRequest {
  type: "Chat";
  text: string;
}

export type ClientMessage = AuthRequest | ChatSendRequest;
```

Important: These types exist only at compile time. Data arriving from TCP is just bytes/text. `JSON.parse()` returns runtime data that can be malformed, so you still need runtime validation such as `asAuthRequest()` and `asChatSendRequest()`.

```
private asAuthRequest(value: unknown): AuthRequest | undefined {
  if (!value || typeof value !== "object") return undefined;
  const v = value as Record<string, unknown>;

  if (
    (v.type === "Login" || v.type === "Register") &&
    typeof v.userName === "string" &&
    typeof v.password === "string"
  ) {
    return v as unknown as AuthRequest;
  }

  return undefined;
}
```

This is a security and reliability habit worth keeping. A TypeScript interface cannot protect you from untrusted network input by itself.

4. TCP byte streams, framing and LineReader

This was one of the hardest conceptual parts because TCP does not send "messages". TCP gives you a continuous ordered stream of bytes. One write on the sender does not guarantee one matching data event on the receiver.

```
Sender writes:
{"type":"Chat","text":"Hello"}

Receiver might observe chunks like:
{"type":"Cha
t","text":"Hello"}
```

```
or several messages in one chunk:  
msg1  
msg2
```

Your solution is newline-delimited JSON. Each complete JSON message ends with a newline. `LineReader` buffers incoming text until it finds a newline, then yields exactly one complete line.

```
public async *lines(): AsyncGenerator<string> {  
    // receive arbitrary chunks  
    // append to buffer  
    // find newline  
    // extract one complete line  
    // yield line  
}
```

Memory rule: TCP transports bytes. Your application protocol defines message boundaries. In this project, newline is the boundary.

Async generators and yield

An async generator produces values over time. The server does not know when the next client message will arrive, so the sequence is asynchronous.

```
for await (const line of connection.lines()) {  
    const parsed = JSON.parse(line);  
    // handle this complete message  
}
```

Read it as: "for each complete line that eventually arrives from this connection, wait for it and process it." This is close to C# `await foreach` over an `IAsyncEnumerable<string>`.

yield: `yield` does not print. It hands one value to the consumer and pauses the generator until the consumer asks for the next value.

5. ClientConnection and socket lifecycle

ClientConnection is a wrapper around one Node Socket. It keeps networking details out of ChatServer and gives the server higher-level operations such as lines(), send(), close() and destroy().

Operation	Meaning	Typical use
socket.end()	Graceful shutdown of writable side; pending writes can finish	Normal disconnect
socket.destroy()	Immediate teardown	Cleanup/failsafe after an error
connection.lines()	Returns async sequence of complete text lines	Server receive loop
writeJsonLine(...)	Serialize object + append newline + write to socket	Send framed JSON

C# comparison: Node Socket covers much of what TcpClient + NetworkStream do together. Your custom LineReader provides the line-reading convenience that StreamReader previously hid for you.

6. ChatClient lifecycle

```
public async start(host: string, port: number): Promise<void> {
    this.socket = await this.connect(host, port);
    const lineReader = new LineReader(this.socket);

    const authenticated = await this.authenticate(this.socket, lineReader);
    if (!authenticated) {
        this.socket.end();
        return;
    }

    const receiveTask = this.receiveLoop(lineReader);
    const sendTask = this.sendLoop(this.socket);

    await Promise.race([receiveTask, sendTask]);
    this.socket.end();
}
```

The client has a clear lifecycle: create TCP connection → authenticate → run receive/send loops concurrently → finish when one loop ends → close the socket.

Why Promise.race()

receiveLoop() waits for messages from the server while sendLoop() waits for user input. Both need to be active at the same time. Promise.race() completes when the first of those operations ends, for example when the user enters /quit or the server disconnects.

```
const receiveTask = this.receiveLoop(lineReader);
const sendTask = this.sendLoop(this.socket);
await Promise.race([receiveTask, sendTask]);
```

Concurrency model: A Promise is not a thread. Node can keep multiple I/O operations in flight using the event

loop. Your code can await network input and console input without creating one thread per operation.

Socket | undefined

You declared the field as optional because a ChatClient can exist before start() creates a connection:

```
private socket: Socket | undefined;
```

After this assignment, TypeScript can narrow the value inside start():

```
this.socket = await this.connect(host, port); // connect returns Promise<Socket>  
const lineReader = new LineReader(this.socket); // now known to be Socket
```

Important syntax: this.socket?.end() is optional chaining: call end only if socket exists. this.socket? by itself inside a function argument is invalid syntax and caused an "Expression expected" error during the project.

7. AuthenticationService, PBKDF2 and UserStore

AuthenticationService owns the decision between registration and login. ChatServer should not duplicate those details.

```
public async authenticate(request: AuthRequest): Promise<AuthResponse> {
  return request.type === "Register"
    ? this.register(request.userName, request.password)
    : this.login(request.userName, request.password);
}
```

Registration flow

14. Load all existing users from UserStore.
15. Check whether the username already exists using a case-insensitive comparison.
16. Hash the password using PBKDF2 with a random salt.
17. Store only userName, salt and passwordHash - not the plaintext password.
18. Save the updated user collection back to users.json.
19. Return AuthResponse success=true.

```
const passwordRecord = await hashPassword(password);

users.push({
  userName,
  salt: passwordRecord.salt,
  passwordHash: passwordRecord.passwordHash
});

await this.userStore.saveAll(users);
```

Login flow

20. Load users.
21. Find matching username.
22. If no user exists, return Invalid credentials.
23. Re-run PBKDF2 using the supplied password and the stored salt.
24. Compare derived hash with stored hash using a timing-safe comparison.
25. Return success or failure.

Password security model: The server does not decrypt a stored password. It derives a hash from the attempted password and compares that derived value with the stored passwordHash.

UserStore and valid JSON

A major runtime error came from users.json being an empty file. JSON.parse() cannot parse an empty string.

```
JSON.parse("") // error: Unexpected end of JSON input
JSON.parse("[]") // valid empty array
```

Memory rule: An empty file is not valid JSON. An empty JSON array is exactly: []

That is why the initial users.json should contain only:

```
[ ]
```

8. ChatServer orchestration and MessageDispatcher

ChatServer is the central coordinator. It accepts the socket, wraps it in ClientConnection, validates messages, calls AuthenticationService and then routes authenticated chat messages to MessageDispatcher.

```
private async handleClient(socket: Socket): Promise<void> {
  const connection = new ClientConnection(socket);
  let authenticated = false;

  try {
    for await (const line of connection.lines()) {
      const parsed: unknown = JSON.parse(line);

      if (!authenticated) {
        // validate AuthRequest
        // call authService.authenticate(request)
        // send AuthResponse
        // only then mark authenticated
        continue;
      }

      // validate ChatSendRequest
      // dispatcher.chat(...)
    }
  } finally {
    // remove client
    // broadcast Leave
    // destroy connection
  }
}
```

The authentication bug you found

At one point ChatServer validated that the request looked like an AuthRequest, but then immediately marked the connection authenticated without calling AuthenticationService. This caused the constructor-injected authService to be reported as "declared but never used".

```
// Wrong flow
const request = this.asAuthRequest(parsed);
connection.userName = request.userName;
authenticated = true;

// Correct missing step
const response = await this.authService.authenticate(request);
connection.send(response);

if (!response.success) continue;

connection.userName = request.userName;
authenticated = true;
```

Debugging lesson: An "unused dependency" warning can be an architecture clue. In this case it revealed that an entire authentication step had been skipped.

MessageDispatcher

The dispatcher creates server-side ChatMessage objects and broadcasts them to the active ClientConnection set. ChatServer decides when someone joins, leaves or sends chat; MessageDispatcher decides how that event is distributed.

```
public joined(clients: ReadonlySet<ClientConnection>, name: string): void {
    this.broadcast(clients, {
        type: "Join",
        name,
        text: `${name} joined the chat.`,
        sentAt: new Date().toISOString()
    });
}
```

ReadonlySet: The dispatcher can iterate over clients but should not own membership changes. ChatServer owns adding/removing clients. That is a useful responsibility boundary.

9. TypeScript syntax that mattered most

Syntax	Meaning	Example
const	Binding cannot be reassigned	const name = "Alice"
let	Binding can be reassigned	let authenticated = false
: Type	Explicit type annotation	port: number
A B	Union: A or B	Socket undefined
? on property	Optional / may be undefined	socket?: Socket
?.	Optional chaining	socket?.end()
??	Fallback only for null/undefined	value ?? "default"
⇒	Arrow function / callback	() ⇒ console.log("ok")
Promise<T>	Async result that eventually produces T	Promise<Socket>
async/await	Asynchronous control flow	await this.connect(...)
for...of	Iterate values	for (const client of clients)
for await...of	Iterate async values	for await (const line of lines())
...obj	Spread properties/items	{ ...user, active: true }
\${expr}	Expression interpolation inside backticks	`Hello \${name}`

Template literals: the backtick issue

You initially used C#-like interpolation but with the wrong JavaScript syntax:

```
// Wrong
console.log("[$(time)] *** ${message.text}");

// Correct TypeScript/JavaScript
console.log(`${$(time)} *** ${message.text}`);
```

For normal strings, double quotes are fine. Use backticks when you want \${...} interpolation or a multiline string.

C# mapping: C#: \$"Hello {name}" | TypeScript: `Hello \${name}`

Type vs value: Socket vs socket

```
import { createConnection, type Socket } from "node:net";

private socket: Socket | undefined;

private connect(host: string, port: number): Promise<Socket> {
  const socket = createConnection({ host, port });
  // ...
}
```

Identifier	Role
Socket	Imported type/class name; capitalized

<code>this.socket</code>	ChatClient field holding the current Socket instance
<code>const socket</code>	Local variable inside connect()
<code>Promise<Socket></code>	Promise that resolves to a Socket instance

10. Your main struggles and what each one taught you

The most useful part of this project was not avoiding errors; it was learning what category each error belonged to. Several issues looked unrelated at first but were actually one root cause.

Struggle	Root cause	Lesson to keep
npm run build → Missing script	package.json had no build script	npm only runs named scripts that exist; add "build": "tsc"
TS2307 Cannot find module	Relative import path or wrong file location	export permission and import path are separate concerns; ./ means same folder
Many socket red lines at once	Malformed connect() signature	Fix the earliest parser/compiler error first; later errors may be cascades
Promise<socket> error	Missing colon plus wrong capitalization	Return type syntax is): Promise<Socket>
this.socket? caused Expression expected	? used outside valid optional syntax	Use this.socket?.end(); or pass this.socket after narrowing
authService declared but never used	Server skipped actual authentication	Warnings can expose missing architecture, not just cosmetic cleanup
Unexpected end of JSON input	users.json was blank	Blank file != []; JSON.parse needs valid JSON text
Register gave Invalid credentials	Client expected choice exactly "2"	Trace the value entering a ternary; UI text and accepted input must match
String interpolation printed wrong syntax	Used \${...} instead of `\${...}` and normal quotes	Template literal requires backticks and `\${expression}`
Client seemed stuck after failed auth	Socket can close while readline remains open	Processes can stay alive because another resource/event handle is still active
Date shown instead of time	Used toLocaleDateString()	Pick formatter based on display intent: date vs time

The cascading compiler error example

This was one of the best debugging lessons in the project. You had eight errors, many mentioning socket, but the real root cause was one method declaration:

```
// Broken
private connect(host: string, port: number) Promise<socket> {

// Correct
private connect(host: string, port: number): Promise<Socket> {
```

The missing colon meant TypeScript could not correctly parse the method signature. The lowercase socket also referred to no valid type. Because later code depended on connect() returning Socket, many unrelated-looking errors disappeared after fixing this one line.

Debugging rule: When the compiler reports many errors, repair the earliest syntax/type error and rerun the checker before touching later lines.

11. Debugging stack traces and compiler cascades

Compiler errors: read location + code first

```
src/client/main.ts:2:39 - error TS2307
Cannot find module './authenticationService.js'
```

The compiler gave the exact file, line and column. That told us the failing file was `src/client/main.ts`, even though the `server/main.ts` imports looked correct. The key habit is to trust the exact path printed by the compiler.

Runtime stack traces: start at the top failure, trace calls downward

```
SyntaxError: Unexpected end of JSON input
    at JSON.parse
    at UserStore.getAll (...userStore.ts:18:25)
    at AuthenticationService.login (...authenticationService.ts:40:23)
    at ChatServer.handleClient (...chatServer.ts:48:34)
```

Interpretation: `JSON.parse` threw. It was called by `UserStore.getAll`. `login()` called `getAll()`. `ChatServer` called `login` through `authentication`. The deepest useful source line was `UserStore.getAll` at line 18, where the invalid file content entered `JSON.parse()`.

```
ChatServer.handleClient()
  -> AuthenticationService.login()
    -> UserStore.getAll()
      -> JSON.parse("")
        -> crash
```

Stack-trace rule: The first line tells you what failed. The following frames tell you how execution reached that failure. Find the first frame that belongs to your own code and inspect the data entering that line.

A repeatable debugging loop

26. Run `npm run check` before runtime testing. Fix compile/type errors first.
27. If there are many errors, fix the earliest one and rerun instead of editing all red lines.
28. For runtime errors, read the stack trace and identify the first line in your own source.
29. Inspect the actual runtime value near that line with a small temporary `console.log` when necessary.
30. Trace the value backward: where was it created, transformed, parsed or read from disk/network?
31. Fix the root cause, remove temporary logging, rerun `npm run check`, then retest end-to-end.

Why this scales: The same loop works for Node REST APIs: `request` → `route` → `validation` → `service` → `database`. It also maps well to C# stack traces and ASP.NET request pipelines.

12. Final end-to-end data flow

Register

```
User chooses 2
-> ChatClient builds AuthRequest(type="Register")
-> writeJsonLine(socket, request)
-> TCP newline-delimited JSON
-> ChatServer LineReader yields one complete line
-> JSON.parse
-> asAuthRequest runtime validation
-> authService.authenticate(request)
-> register(userName, password)
-> UserStore.getAll()
-> hashPassword(password)
-> saveAll(users)
-> AuthResponse(success=true)
-> connection.send(response)
-> client reads AuthResponse
-> client enters send/receive loops
```

Send a chat message

```
Console input: "hello"
-> ChatClient sendLoop()
-> { type: "Chat", text: "hello" }
-> writeJsonLine()
-> TCP
-> ChatServer parses + validates ChatSendRequest
-> dispatcher.chat(clients, userName, text)
-> broadcast()
-> each ClientConnection.send()
-> each client receiveLoop()
-> JSON.parse as ChatMessage
-> console.log formatted message
```

Disconnect

```
/quit
-> sendLoop returns
-> Promise.race finishes
-> client socket.end()
-> server loop ends / finally runs
-> clients.delete(connection)
-> dispatcher.left(...)
-> remaining clients receive Leave message
-> connection.destroy() cleanup
```

Architecture summary: The client owns user interaction. The server owns trust and session state. AuthenticationService owns credentials. UserStore owns persistence. MessageDispatcher owns broadcasting. Shared types define the protocol.

13. Memory rules and next-step practice

Topic	Memory rule
TCP	It is a byte stream; your protocol creates message boundaries.
Newline JSON	Serialize JSON + newline; reader buffers until newline.
async generator	Produces values over time; consume with for await...of.
Promise<T>	Eventually produces T; comparable to Task<T> at application level.
Socket import	Importing Socket gives you the type/class, not an already-connected instance.
Optional chaining	obj?.method() means call only if obj exists.
Runtime validation	Interfaces disappear at runtime; validate network input.
Authentication	Validate shape first, then verify credentials, then mark authenticated.
Hashing	Store salt + derived hash, never plaintext password.
JSON storage	Blank file is invalid; [] is a valid empty array.
Compiler errors	Fix the earliest root error and rerun.
Stack trace	Find the first failing line in your own code and inspect the input value.
Template literal	Backticks + \${expression}; normal strings can stay in double quotes.

What to carry into your REST API week

The TCP-specific pieces will change, but the architecture you learned is reusable. In a REST API, HTTP replaces your raw TCP protocol and route handlers replace your manual message dispatcher.

TCP chat	REST API
-----	-----
LineReader / JSON framing	-> HTTP request parsing handled by framework
AuthRequest	-> Login/Register request DTO
ChatServer	-> Router/controller + application orchestration
AuthenticationService	-> Authentication service
UserStore	-> Repository / PostgreSQL access
Message validation	-> Request schema validation
AuthResponse	-> HTTP response + status code
Client ownership/session	-> JWT/session + authorization

Recommended small exercises

32. Add strict validation so only 1 or 2 is accepted at the Login/Register menu.
33. Close the readline interface cleanly when the client exits so Ctrl+C is not needed after a failed authentication attempt.
34. Change the display timestamp from toLocaleDateString() to a chat-friendly time format.
35. Add a server-side maximum message length and reject empty/oversized chat messages.
36. Add a duplicate-login rule so the same username cannot have two active connections, if you want extra practice.
37. Then stop extending the TCP project and move to REST APIs as planned. Reuse the same service/storage/authentication separation.

Completion checkpoint: You successfully reached a working multi-client chat. The next value comes from transferring these concepts into HTTP/REST rather than adding many more features to the TCP project.

One-page quick reference

TYPE / VALUE	
Socket	= type/class
this.socket	= current instance
Promise<Socket>	= async result containing Socket
ASYNC	
await promise	= wait without blocking the event loop
for await (... of ...)	= async iteration
async * function	= async generator
NULLABILITY	
Socket undefined	= may not exist yet
socket?.end()	= end only if socket exists
STRINGS	
"normal text"	= normal string
`Hello \${name}`	= interpolated template literal
NETWORK	
TCP	= ordered byte stream
newline	= your message boundary
JSON.parse	= text -> runtime value
runtime validator	= proves untrusted value has expected shape
DEBUGGING	
npm run check	= type-check without emitting JS
npm run build	= compile with tsc
first compiler error	= fix this before chasing later errors
stack trace	= failure + call path to failure

Project status: COMPLETE

You now have a concrete TypeScript example of sockets, async iteration, runtime validation, password hashing, persistence, service boundaries and multi-client broadcasting. Those concepts are strong preparation for your next step: several small REST APIs, then React.